

p1099R0

Using Enum

Draft Proposal, 23 May 2018

This version:

<https://github.com/atomgalaxy/using-enum/using-enum.bs>

Authors:

Gašper Ažman <gasper.azman@gmail.com>

Jonathan Müller <jonathan.mueller@foonathan.net>

Audience:

SG1, LEWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Abstract

Class enums are restricted namespaces. Let's extend the using declaration to them.

Table of Contents

1	Motivation
2	Rationale
2.1	Consistency
2.2	Better Identifiers
2.3	Evidence of Need
3	Proposal
4	<code>using enum IDENTIFIER</code>
5	<code>using ENUM_ID::IDENTIFIER</code>

§ 1. Motivation

Consider an enum class:

```
enum class rgba_color_channel { red, green, blue, alpha };
```

Currently, a switch using this enum looks as follows:

```
std::string_view to_string(rgba_color_channel channel) {  
    switch (channel) {  
        case rgba_color_channel::red:    return "red";  
        case rgba_color_channel::green:  return "green";  
        case rgba_color_channel::blue:   return "blue";  
        case rgba_color_channel::alpha:  return "alpha";  
    }  
}
```

The necessary repetition of the class enum name reduces legibility by introducing noise in contexts where the enum class is obvious.

To eliminate the noise penalty for introducing long (but descriptive) enum class names, this paper proposes that the statement

```
using enum rgba_color_channel;
```

introduce the enum member identifiers into the local scope so that they may be referred to unqualified.

Furthermore, the syntax

```
using rgba_color_channel::red;
```

should bring the identifier `red` into the local scope, so that it may be used unqualified.

The above example would then probably be written as

```
std::string_view to_string(rgba_color_channel channel) {  
    switch (my_channel) {  
        using enum rgba_color_channel;  
        case red:    return "red";  
        case green:  return "green";  
        case blue:   return "blue";  
        case alpha:  return "alpha";  
    }  
}
```

§ 2. Rationale

§ 2.1. Consistency

`enum class`es are not classes - they seem to be closer to namespaces consisting of `static constexpr` inline variables. The familiar `using` syntax that works for namespaces should therefore apply to them as well.

§ 2.2. Better Identifiers

The introduction of this feature would allow better naming of enums. Currently, enums are named with as short an identifier as possible, often to the point of absurdity, when they are reduced to completely nondescriptive abbreviations that only hint at their proper meaning (just what does `zfqc::add_op` mean?)

With this feature, identifiers become available to unqualified lookup in local contexts where their source is obvious, giving control of lookup style back to the user of the enum, instead of baking name semantics into the type of the enum.

§ 2.3. Evidence of Need

At a casual search, we were able to locate this [thread on stackoverflow](#).

§ 3. Proposal

§ 4. `using enum IDENTIFIER`

We propose the addition of a new `using enum` statement:

```
using enum IDENTIFIER;
```

The above statement introduces the members of enum `IDENTIFIER` into the local namespace, thus enabling lookup without qualification. As usual, a name lookup ambiguity makes the program ill-formed (diagnostic required).

§ 5. `using ENUM_ID::IDENTIFIER`

We propose to allow the syntax of

```
using ENUM_ID::IDENTIFIER
```

to introduce the `IDENTIFIER` into the local namespace, aliasing `ENUM_ID::IDENTIFIER`.

§ 6. Acknowledgements

The authors would like to thank Marcel Ebmer and Lisa Lipincott for early feedback.